# I swear it worked on my computer! Computational Reproducibility Using Nix and rix

Jason Geller, PH.D.(he/him)

2026-11-11

# Slides



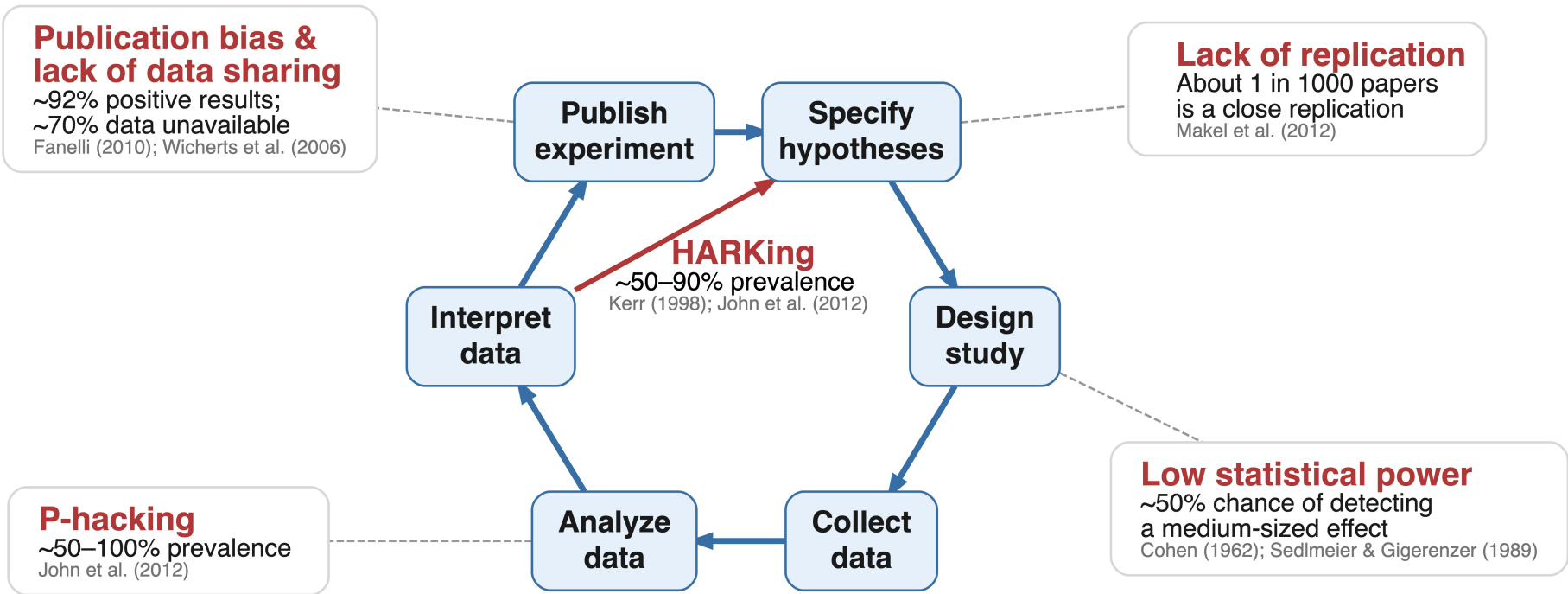
Slides: jgeller112.github.io/princeton_2026_rix-nix

# Disclaimer

I will mostly be using  examples.

I have no connection to Nix or `{rix}`: just a happy user.

# The Replication Crisis[1]



# ~~Replication Crisis~~ Credibility Revolution!

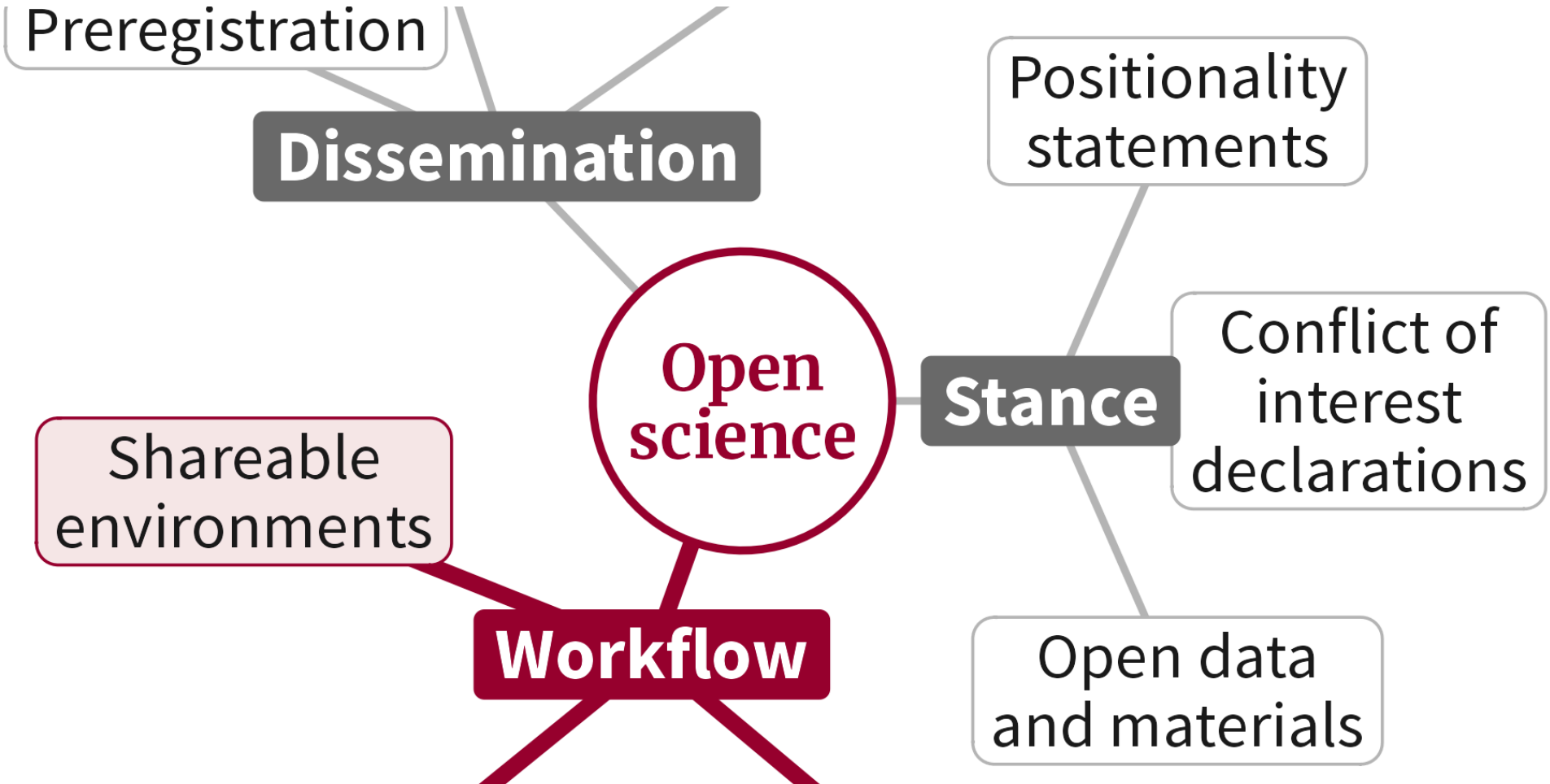


# Open Science

- Open science: being open and transparent at every stage of research[1]
- Three parts:
  - → Dissemination
  - → Stance
  - → *Workflow*: *Could someone else rerun your whole pipeline and get your results?*

[1] Figure adapted from Casillas, J. V., Constantin-Dureci, G., Andreu Rascón, I., et al. (2025). Opening open science to all: Demystifying reproducibility and transparency practices in linguistic research. *Linguistics*, *63*(6), 1547–1575. https://doi.org/10.1515/ling-2023-0249

# Open Science



# What is Reproducibility?

| Analysis / Data | Same Data | Different Data |
|---|---|---|
| Same Analysis | **Reproducible** | Replicable |
| Different Analysis | Robust | Generalisable |

# Demo: everything is shared

```
study1/
├── analysis.R
└── data/
    └── data_final.csv
```

```r
# analysis.R
dat ← read.csv("data/data_final.csv")

cohens_d ← function(x) {
  m ← tapply(x$recall, x$condition, mean)
  s ← sqrt(mean(tapply(x$recall, x$condition, var)))
  unname((m["retrieval"] - m["control"]) / s)
}

set.seed(2018)
boot ← replicate(2000, cohens_d(dat[sample(nrow(dat), replace = TRUE), ]))

round(cohens_d(dat), 2)
round(quantile(boot, c(.025, .975)), 2)
```

Paper (2019): $d = 0.45$, 95% bootstrap confidence interval (CI) [0.01, 0.91], which excludes zero

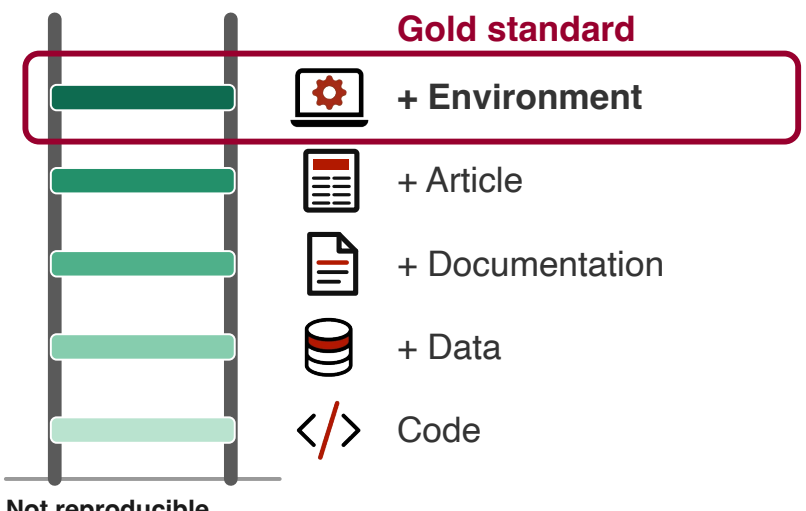
# Demo: same code, different year

| | 2019: author (R 3.5.3) | 2026: me (R 4.6.1) |
|---|---|---|
| *d* | 0.45 | 0.45 |
| 95% CI | [0.01, 0.91] | [**−0.02**, 0.88] |
| CI includes 0? | No | **Yes** |

Why? R 3.6.0 changed how `sample()` draws, so the same seed gives different bootstrap samples.

# Computational Reproducibility

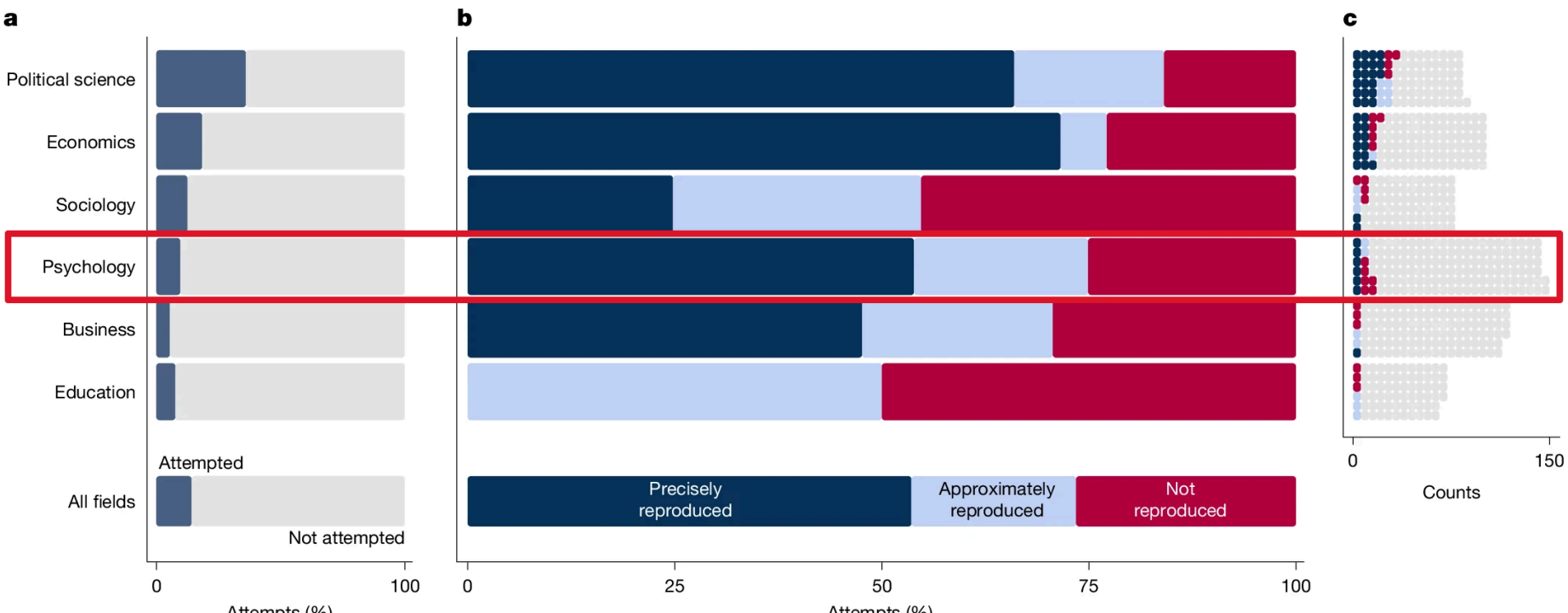
"Ability to obtain consistent computational results using the same input data, computational steps, methods, code, and conditions of analysis"[1]




[1] Schultze, A., Saul, L., Roesch, E., et al. (2026). *Computational reproducibility: A primer from UKRN*. Zenodo. https://doi.org/10.5281/zenodo.19232891

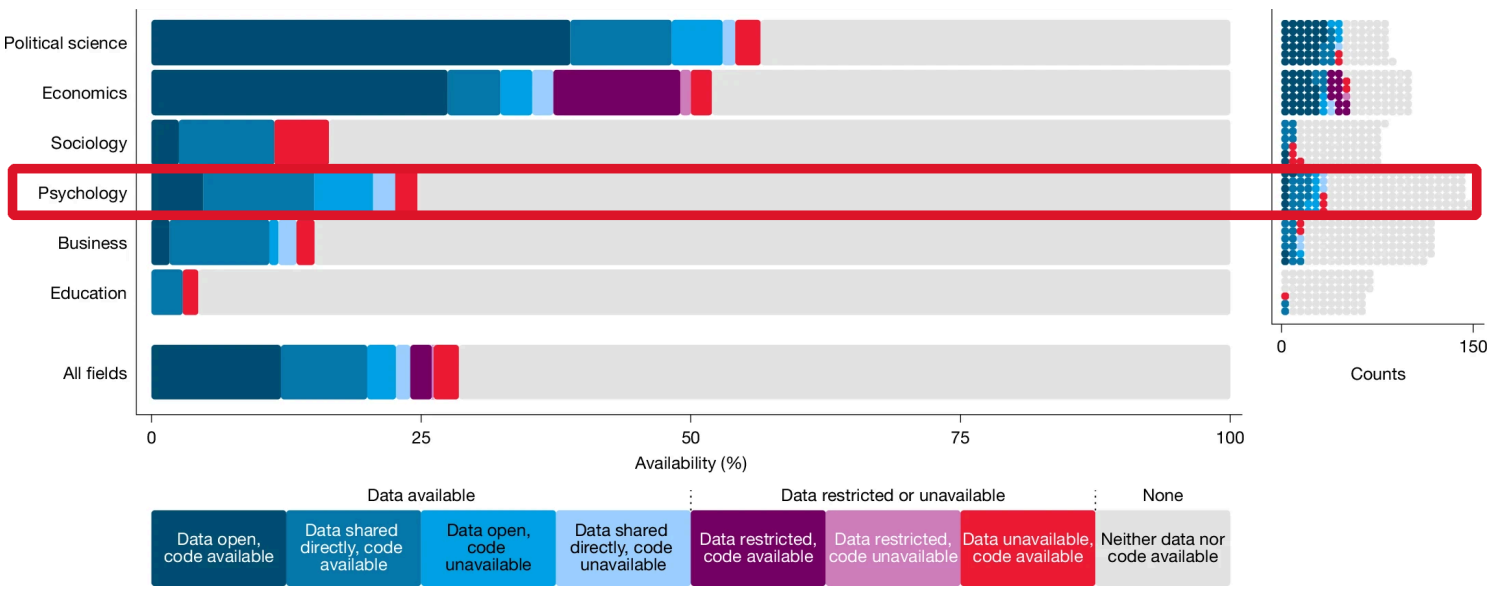
# Psychology: about half reproduce[1]

- SCORE: 600 papers, 62 journals, several fields (2009-2018)
- 144 of 600 shared data; 38 more rebuilt from source data



# Why so few attempts? No data[1]

- Few psychology papers shared both data and code

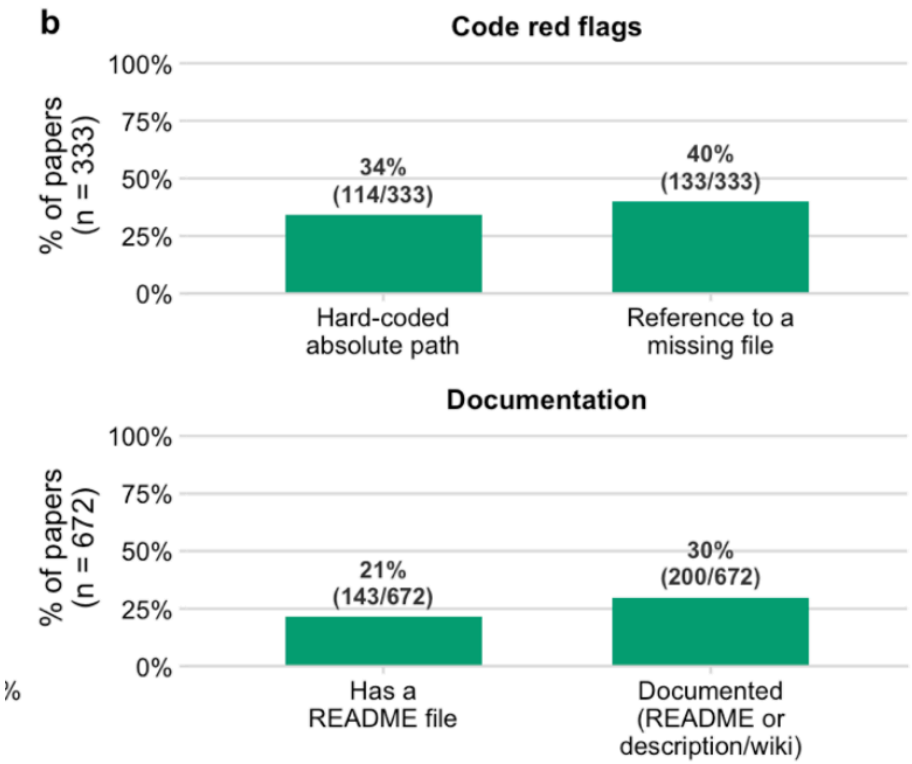


# Shared ≠ usable

- Quintana (2026)[1]
- Downloaded materials from OSF repos (334 papers)
  - → *Psych Sci*, *JML*, and *ESP*
  - → When materials are shared, are they usable?

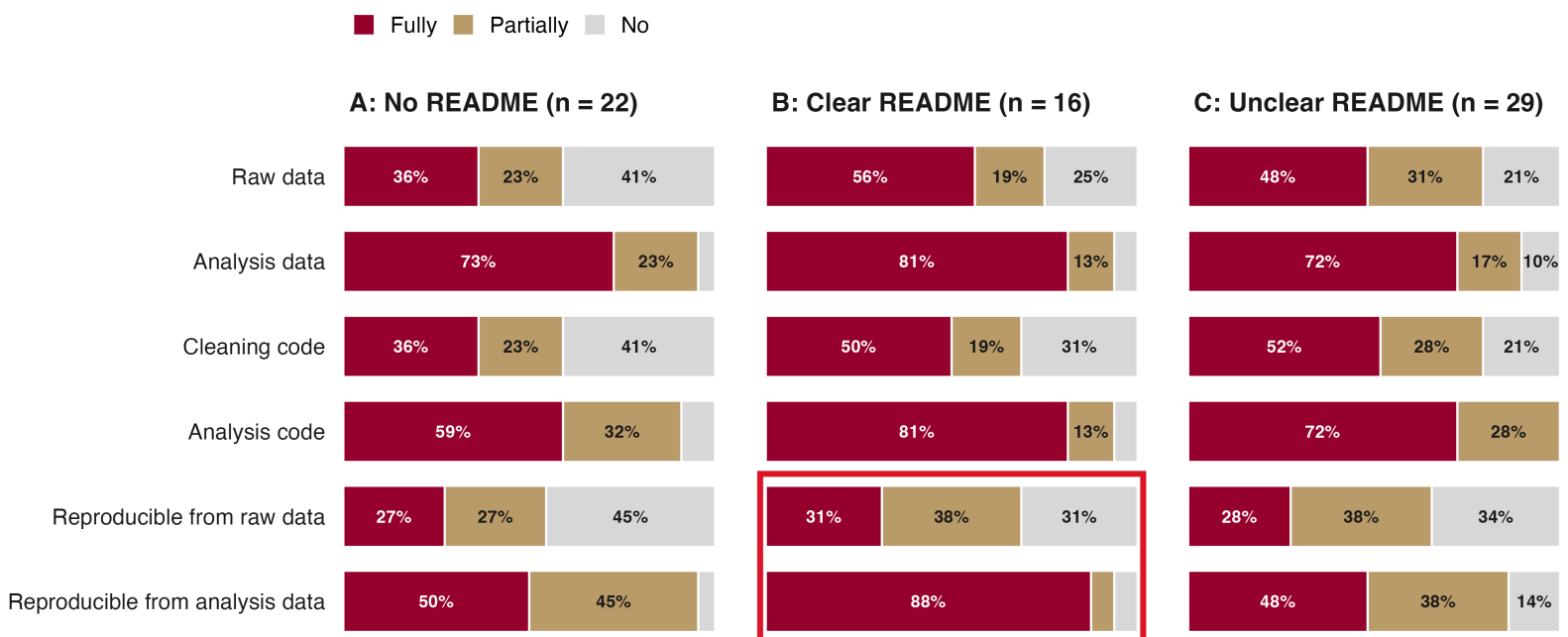
[1] Quintana, D. S. (2026, September 19). Shared research materials in psychology are widely downloaded but often difficult to reuse. PsyArXiv. https://osf.io/preprints/psyarxiv/yswku_v3

# Shared ≠ usable



# READMEs help, but only so far

- Institute for Replication[1]: 67 *Psych Science* papers (2024-2025)



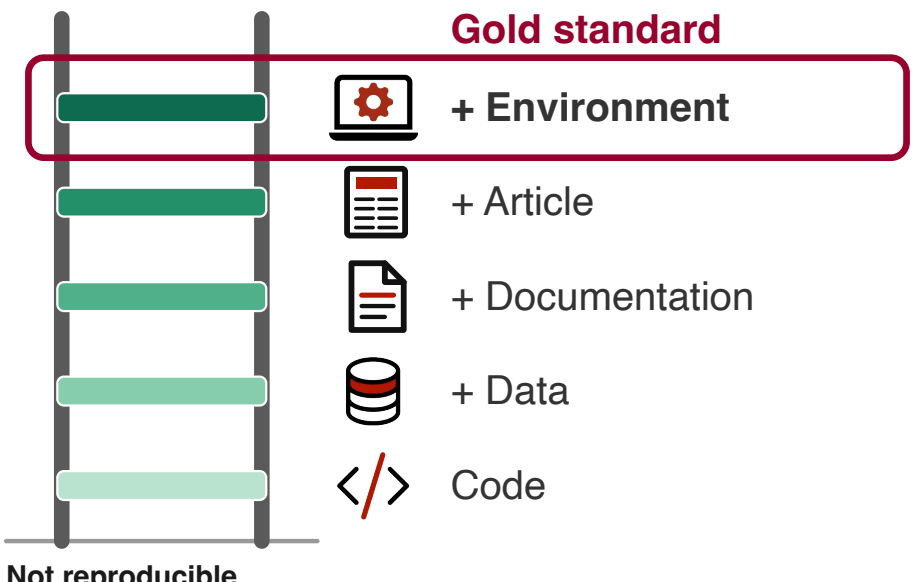

[1] Brodeur, A., Abdul Baki, G., & Fazio, L. (2026). Mass reproducibility in psychological science.

# READMEs help, but only so far

- Clear README: 88% from analysis data, only 31% from raw data

# Psychology: not so good 😟

- Back to the ladder: where do these studies leave us?




- Code, data, and a README get you partway. The environment is the missing step.

# The Computational Environment

# Which “reproducible”?[1]

## Computational environment

| | **across space** (spatial reproducibility) | **across time** (temporal reproducibility) |
|---|---|---|
| **the environment builds** (rebuildability) | builds on another machine or OS, today | the same description still builds years later |
| **the results are the same** (functional reproducibility) | same versions, same numbers, on another machine or OS | same versions, same numbers, years later |
| **the bytes are the same** (bitwise reproducibility) | identical hashes on another machine | identical hashes, years later |

# Environment Fragility

Setting bitwise reproducibility aside, even error-free R code sits on top of four layers, each of which may drift over space and time:

1. R version
2. R packages
3. Document tools: e.g., Pandoc, LaTeX, Quarto
4. System libraries and compilers: e.g., gcc, libpng, BLAS/LAPACK

# 1. R version

- Syntax: the native pipe `|>` exists only since R 4.1.0.
- Defaults: R 4.0.0 stopped turning strings into factors. Same code, different object:

```
df ← data.frame(group = c("a", "b"))
class(df$group); as.numeric(df$group)
#> R 3.6.3:  "factor"      1  2   (stringsAsFactors = TRUE)
#> R 4.0.0+: "character"  NA NA   (stringsAsFactors = FALSE)
```

- Random numbers: R 3.6.0 changed how `sample()` draws. Same seed, different draws:

```
set.seed(123); sample(1:10, 3)
#> R 3.5.3:   3  8  4    (sample.kind = "Rounding")
#> R 3.6.0+:  3 10  2    (sample.kind = "Rejection")
```

# 2. R packages

- Renamed or deprecated: `dplyr::cur_data()`, but now `dplyr::pick()` (warning about the future).
- Internal functions (discouraged, but used nonetheless):

```
mypkg:::fit_model(df)   # internal helper
#> mypkg 1.0: works
#> mypkg 2.0: Error: object 'fit_model' not found
#>            (renamed to fitModel(), no warning)
```

# 3. Document tools

- Pandoc and LaTeX, called by Quarto (also outside R) or by `{rmarkdown}`. Each has its own version.
- From this talk's demo: HTML fine, PDF failed until one more LaTeX package was declared:

```
LaTeX Error: File `lualatex-math.sty' not found.
```

# 4. System libraries and compilers

- Many R packages are compiled code (C, C++, Fortran) and use libraries from the OS (e.g., libpng, BLAS/LAPACK).
  - → Today: CRAN hands you a binary. However, years later: only the source. Your machine compiles it, with today's compiler and today's libraries.
- Same script, different BLAS: an eigenvector sign flip in the system's BLAS/LAPACK changes `MASS::mvrnorm()` draws (`mvtnorm::rmvnorm()` is fine). Based on an illustration by Danielle Navarro.

```
== MASS (native R) ==
  macOS:    0.28780065222640439   0.59425558497432929  -0.25961849316812602  -2.3918999759288693
  Ubuntu:  -0.43918332712576574   0.25608933342594919   0.85420518537273971  -2.2883238259260961
  Windows: -0.43918332712576580   0.25608933342594914   0.85420518537273971  -2.2883238259260965
  ⇒ DIVERGE
```

# Python, too: language version

- Syntax: `match` statements exist only since Python 3.10, the walrus `:=` since 3.8.
- Random numbers: sampling from a set works in 3.10, but the draws depend on set order, which changes *every run*. In 3.11+ it errors:

```
import random; random.seed(1); random.sample({"a", "b", "c"}, 2)
#> Python 3.10:  ['a', 'b']   (or ['c', 'a'], ... depends on the run)
#> Python 3.11+: TypeError: Population must be a sequence.
```

# Python, too: packages

- Changed defaults, loud: pandas 2.0 stopped silently dropping text columns:

```
df = pd.DataFrame({"group": ["a", "a", "b"],
                   "subject": ["s1", "s2", "s3"], "rt": [500, 520, 610]})
df.groupby("group").mean()
#> pandas 1.5: mean rt per group    (subject column quietly dropped)
#> pandas 2.0+: TypeError: agg function failed [how→mean,dtype→object]
```

- Changed defaults, silent: numpy 2.0 changed type promotion. No error, different number:

```
x = np.float32(0.1) + 0.2
#> numpy 1.26: 0.30000000149011613   (float64)
#> numpy 2.0+: 0.3                   (float32)
```

- Removed: `np.float` was removed in numpy 1.24, `DataFrame.append()` in pandas 2.0.
- Models: scikit-learn 0.22 changed `LogisticRegression`'s default solver (`liblinear` → `lbfgs`): same call, different coefficients.

# Python, too: packages

- System libraries: numpy links OpenBLAS, MKL, or Apple Accelerate depending on how it was installed: the same BLAS story as R.

# Managing the environment



# Managing the environment

`{renv}` R packages

# Managing the environment



# Managing the environment

`{rig}` R version



uv Python + packages




conda + compiled libraries



Docker the OS

# Managing the environment

Each tool pins one piece: many tools, many skill sets.

# {renv}

- Most popular package for reproducibility, and very easy to use:
  - → `renv::init()` snapshots the packages used (and their versions) into `renv.lock`
  - → `renv::restore()` reinstalls them
- But it records, and does not restore, the version of R (can use {rig})
- But installing old packages can fail (missing OS dependencies)

# {renv}



# {renv} in action

```
# analysis.R: palmerpenguins, ggplot2
renv::init()
```

```
- Linking packages into the project
  library ... Done!
The following package(s) will be
updated in the lockfile:

# CRAN ------------------------------
- cli              [* → 3.6.6]
- ggplot2          [* → 4.0.3]
- palmerpenguins   [* → 0.1.1]
- rlang            [* → 1.3.0]
   ... 15 more dependencies

The version of R recorded in the lockfile
will be updated:
- R                [* → 4.6.1]

- Lockfile written to "renv.lock".
```

## renv.lock (excerpt)

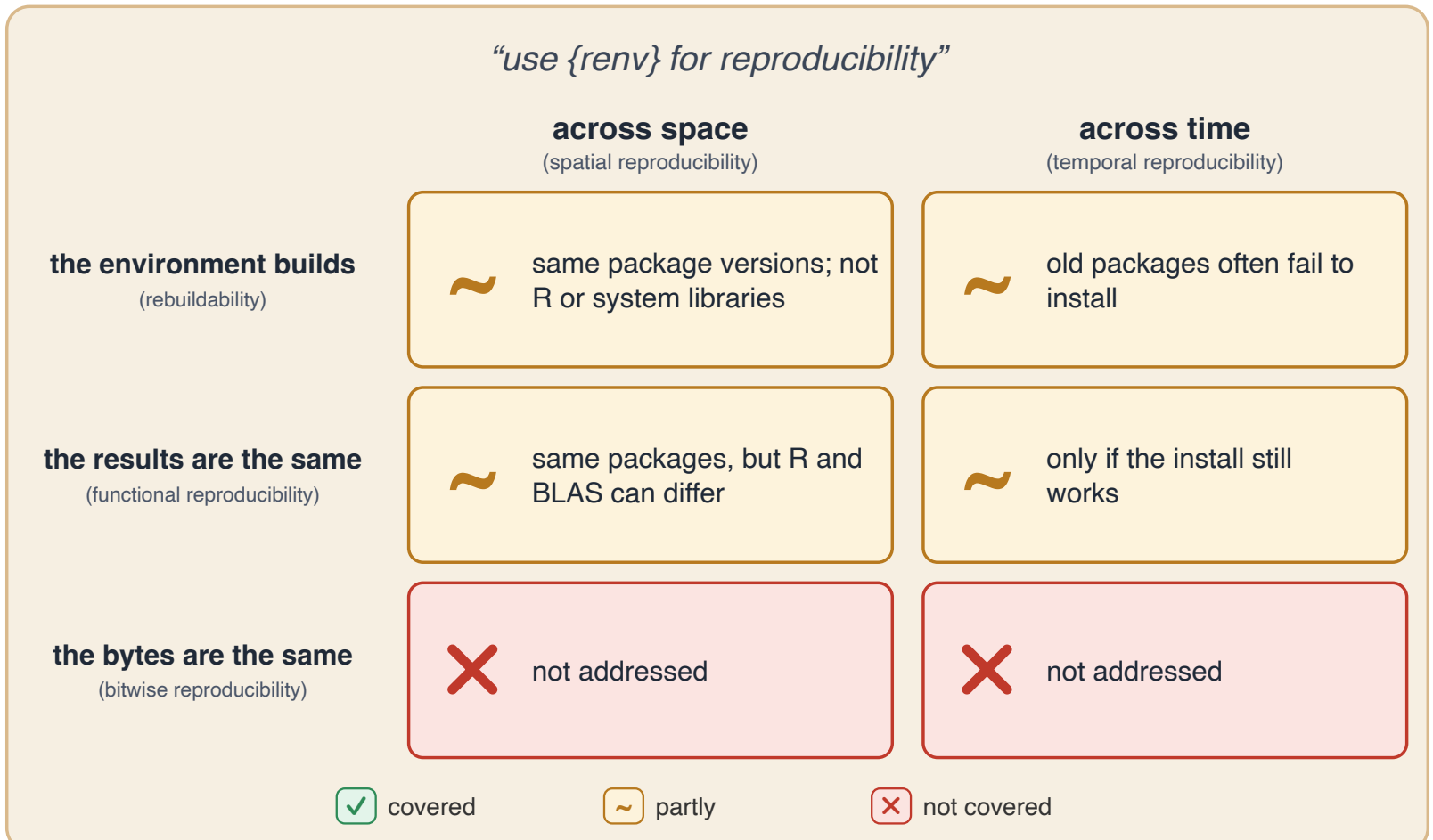
```
{
  "R": {
    "Version": "4.6.1",
    "Repositories": [ ... ]
```

# {renv} in action

```
  },
  "Packages": {
    "ggplot2": {
      "Package": "ggplot2",
      "Version": "4.0.3",
      "Source": "Repository",
      "Repository": "CRAN"
    },
    ...
```

R 4.6.1 is written down, not installed. `renv::restore()` uses whatever R you have.

# {renv}: which boxes?



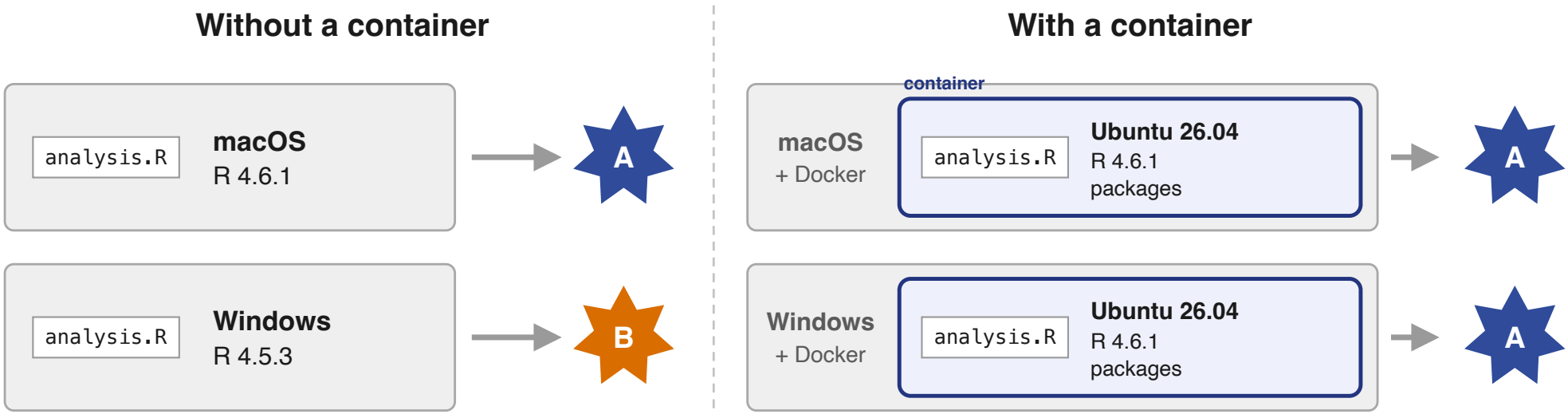
# Docker: what a container does

Dockerfile (recipe) $\rightarrow$ image (the environment, frozen) $\rightarrow$ container (the image, running)[1]



[1] Figure adapted from Rodrigues, B. *Building reproducible pipelines with R, Docker and Nix*. R-Ladies Rome (English).

# Docker: what a container does




*different setups, (potentially) different outputs*

*the container brings its own OS and R: same output*

Docker images:

- hold all the software and code

# Docker: what a container does

- immutable once built
- shareable on- and offline

# Docker: built for shipping

- Docker's job: make software run the same on a laptop and a server today. Reproducing a result years later was never the goal.

The recipe drifts: every line fetches what's current *on build day*

```
FROM rocker/r-ver:latest                  # which R? today's "latest"
RUN apt-get update && apt-get install -y libxml2-dev   # today's Ubuntu
RUN Rscript -e 'install.packages("brms")'  # today's CRAN
```

# Docker: the catches

1. Archive the image
   - Often several GB; tags on Docker Hub can move or vanish
2. Same image, different CPU
   - Runs on the host's processor; OpenBLAS picks code per CPU, so last digits can differ
3. Costs
   - Linux know-how, slow emulation on Apple Silicon, often banned on university clusters, one company in charge

# Docker: which boxes?

*“Docker guarantees full computational reproducibility”*

| | **across space** (spatial reproducibility) | **across time** (temporal reproducibility) |
|---|---|---|
| **the environment builds** (rebuildability) | ✓ pull the saved image anywhere | ~ drifts unless you pin the base image + use renv or Nix inside |
| **the results are the same** (functional reproducibility) | ✓ same image, same versions | ~ only if you archived the image |
| **the bytes are the same** (bitwise reproducibility) | ~ CPU-dependent BLAS can differ | ✗ not addressed |

✓ covered ~ partly ✗ not covered

# Nix/{Rix}

- Nix is a package manager (+language +build tool):
  - → it installs and manages R/Python packages and software/system dependencies
  - → Built for reproducible builds: R, R packages *and* system dependencies, all declared in one text file (a *Nix expression*)
  - → Every input is pinned by hash: the same expression gives the same environment, on any machine (everything lives in its own folder)
    - The "+ Environment" step of the gold standard

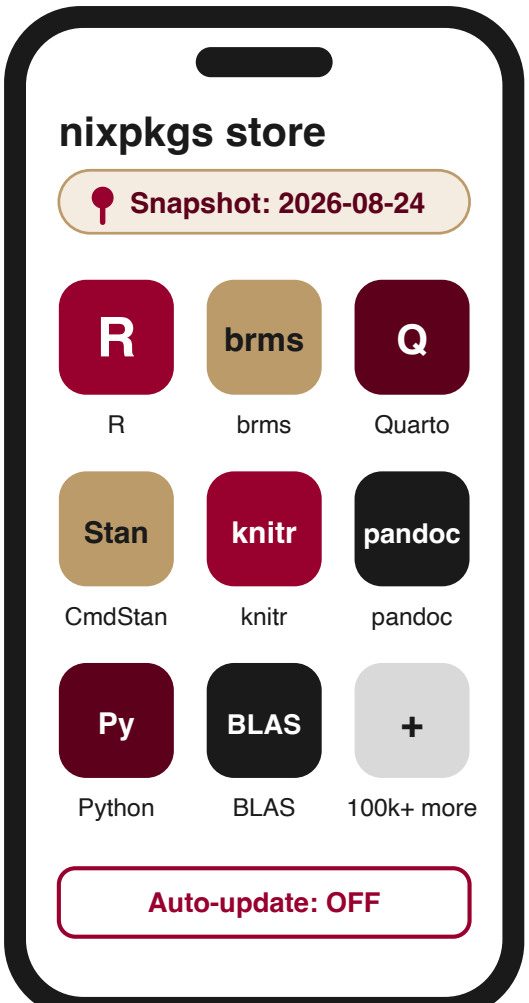


Logo: NixOS artwork (CC-BY 4.0)

# Nix is like an app store

- Like an app store on your phone, but for research software
- One catalog (`nixpkgs`): R, R packages, Quarto, CmdStan, system libraries
- Your `default.nix` is the list of “apps” to install
- The twist: the store is frozen on a date. No auto-updates, so everyone gets the exact same versions

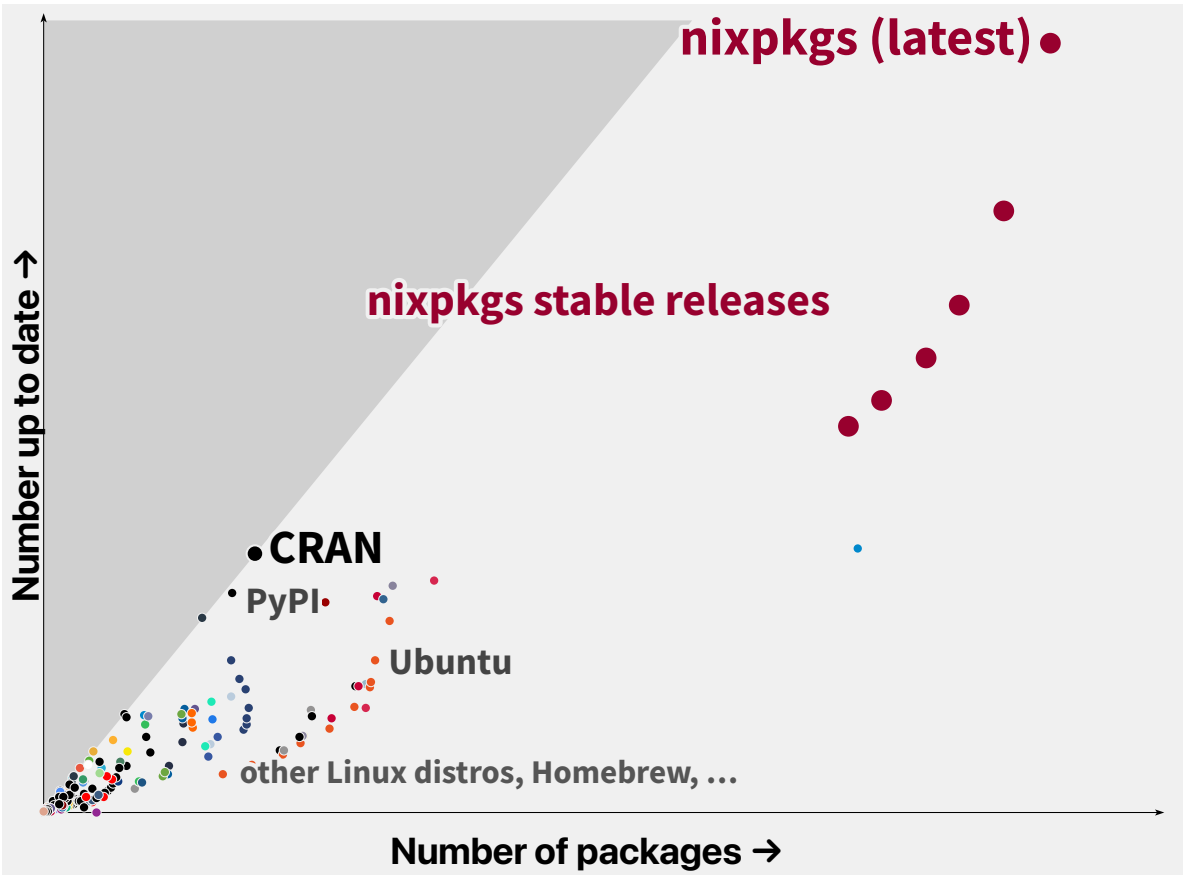
# Nix is like an app store



# How big is `nixpkgs`?[1]

- `nixpkgs` (top right): biggest, freshest; nearly all of CRAN (search)

# How big is `nixpkgs`?[1]



# Nix: `default.nix`

```
let
  pkgs = import (fetchTarball
    "https://github.com/rstats-on-nix/nixpkgs/archive/2026-08-24.tar.gz") {};
  rpkgs = builtins.attrValues {
    inherit (pkgs.rPackages) brms knitr palmerpenguins rmarkdown;
  };
  cmdstanr = pkgs.rPackages.buildRPackage {
    name = "cmdstanr";
    src = pkgs.fetchgit {
      url    = "https://github.com/stan-dev/cmdstanr";
      rev    = "da99e2ba954658bdad63bffb738c4444c33a4e0e";
      sha256 = "sha256-wXfOxBexnuL83fvCM+6qv6d7UhTLGq+Xhvja3lRfQpI=";
    };
  };
  system_packages = builtins.attrValues {
    inherit (pkgs) cmdstan pandoc quarto R;
  };
  shell = pkgs.mkShell {
    buildInputs = [ cmdstanr rpkgs system_packages ];
  };
in { inherit pkgs shell; }
```

The snapshot: every package definition comes from nixpkgs as it was on 2026-08-24.

# Nix: `default.nix`

```
let
  pkgs = import (fetchTarball
    "https://github.com/rstats-on-nix/nixpkgs/archive/2026-08-24.tar.gz") {};
  rpkgs = builtins.attrValues {
    inherit (pkgs.rPackages) brms knitr palmerpenguins rmarkdown;
  };
  cmdstanr = pkgs.rPackages.buildRPackage {
    name = "cmdstanr";
    src = pkgs.fetchgit {
      url    = "https://github.com/stan-dev/cmdstanr";
      rev    = "da99e2ba954658bdad63bffb738c4444c33a4e0e";
      sha256 = "sha256-wXfOxBexnuL83fvCM+6qv6d7UhTLGq+Xhvja3lRfQpI=";
    };
  };
  system_packages = builtins.attrValues {
    inherit (pkgs) cmdstan pandoc quarto R;
  };
  shell = pkgs.mkShell {
    buildInputs = [ cmdstanr rpkgs system_packages ];
  };
in { inherit pkgs shell; }
```

R packages from that snapshot: exact versions, plus all of their dependencies.

# Nix: `default.nix`

```
let
  pkgs = import (fetchTarball
    "https://github.com/rstats-on-nix/nixpkgs/archive/2026-08-24.tar.gz") {};
  rpkgs = builtins.attrValues {
    inherit (pkgs.rPackages) brms knitr palmerpenguins rmarkdown;
  };
  cmdstanr = pkgs.rPackages.buildRPackage {
    name = "cmdstanr";
    src = pkgs.fetchgit {
      url    = "https://github.com/stan-dev/cmdstanr";
      rev    = "da99e2ba954658bdad63bffb738c4444c33a4e0e";
      sha256 = "sha256-wXfOxBexnuL83fvCM+6qv6d7UhTLGq+Xhvja3lRfQpI=";
    };
  };
  system_packages = builtins.attrValues {
    inherit (pkgs) cmdstan pandoc quarto R;
  };
  shell = pkgs.mkShell {
    buildInputs = [ cmdstanr rpkgs system_packages ];
  };
in { inherit pkgs shell; }
```

cmdstanr from GitHub: exact commit *and* content hash. If GitHub changes, the build fails rather than silently differing. By hand, you compute that hash yourself.

# Nix: `default.nix`

```
let
  pkgs = import (fetchTarball
    "https://github.com/rstats-on-nix/nixpkgs/archive/2026-08-24.tar.gz") {};
  rpkgs = builtins.attrValues {
    inherit (pkgs.rPackages) brms knitr palmerpenguins rmarkdown;
  };
  cmdstanr = pkgs.rPackages.buildRPackage {
    name = "cmdstanr";
    src = pkgs.fetchgit {
      url    = "https://github.com/stan-dev/cmdstanr";
      rev    = "da99e2ba954658bdad63bffb738c4444c33a4e0e";
      sha256 = "sha256-wXfOxBexnuL83fvCM+6qv6d7UhTLGq+Xhvja3lRfQpI=";
    };
  };
  system_packages = builtins.attrValues {
    inherit (pkgs) cmdstan pandoc quarto R;
  };
  shell = pkgs.mkShell {
    buildInputs = [ cmdstanr rpkgs system_packages ];
  };
in { inherit pkgs shell; }
```

System software: R itself, Quarto, pandoc and CmdStan, from the same snapshot.

# Nix: `default.nix`

```
let
  pkgs = import (fetchTarball
    "https://github.com/rstats-on-nix/nixpkgs/archive/2026-08-24.tar.gz") {};
  rpkgs = builtins.attrValues {
    inherit (pkgs.rPackages) brms knitr palmerpenguins rmarkdown;
  };
  cmdstanr = pkgs.rPackages.buildRPackage {
    name = "cmdstanr";
    src = pkgs.fetchgit {
      url    = "https://github.com/stan-dev/cmdstanr";
      rev    = "da99e2ba954658bdad63bffb738c4444c33a4e0e";
      sha256 = "sha256-wXfOxBexnuL83fvCM+6qv6d7UhTLGq+Xhvja3lRfQpI=";
    };
  };
  system_packages = builtins.attrValues {
    inherit (pkgs) cmdstan pandoc quarto R;
  };
  shell = pkgs.mkShell {
    buildInputs = [ cmdstanr rpkgs system_packages ];
  };
in { inherit pkgs shell; }
```

The shell: everything above, bundled into what `nix-shell` opens.

# Nix: the empirical evidence

Malka, Zacchiroli, & Zimmermann (2025)[1] rebuilt 709,816 packages from `nixpkgs` snapshots taken every four months, 2017–2023:

- Rebuildable: 99.7% to 99.9% still build from their description.
- Bitwise reproducible: 69% in 2017, 91% in 2023, rising.
- Functionally reproducible: not measured, but follows for every package that rebuilds. From my understanding, every input is pinned by hash, so a rebuild can fail but cannot succeed with different versions. This is not the same with Docker.

[1] Malka, J., Zacchiroli, S., & Zimmermann, T. (2025). Does functional package management enable reproducible builds at scale? Yes. *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*. https://doi.org/10.1109/MSR66628.2025.00115

# Nix: which boxes?

*“Nix: every input pinned by hash”*

| | **across space** (spatial reproducibility) | **across time** (temporal reproducibility) |
|---|---|---|
| **the environment builds** (rebuildability) | ✓ same default.nix, any machine | ✓ 99.7–99.9% still rebuild |
| **the results are the same** (functional reproducibility) | ✓ same pinned versions everywhere | ✓ same pinned inputs, years later |
| **the bytes are the same** (bitwise reproducibility) | ~ mostly: 91% bitwise in 2023 | ~ rising: 69% (2017) → 91% (2023) |

✓ covered ~ partly ✗ not covered

# Nix/`Rix`

- `{rix}`, by Bruno Rodrigues (website), makes writing Nix expressions easy: one R function, `rix()`




```
install.packages("rix", repos = "https://ropensci.r-universe.dev")
```

# Nix/`Rix`: the `rix()` call

```
library(rix)

rix(
  date   = "2026-08-24",
  r_pkgs = c("brms", "palmerpenguins", "knitr", "rmarkdown"),
  git_pkgs = list(
    package_name = "cmdstanr",
    repo_url     = "https://github.com/stan-dev/cmdstanr",
    commit       = "da99e2ba954658bdad63bffb738c4444c33a4e0e"
  ),
  system_pkgs = c("quarto", "cmdstan"),
  ide          = "none",
  project_path = ".",
  overwrite    = TRUE
)
```

`date`: becomes the snapshot line in `default.nix`, pinning *everything* to one snapshot of nixpkgs. R and every package come in the versions they had on that day, on any machine, forever.

# Nix/`Rix`: the `rix()` call

```
library(rix)

rix(
  date   = "2026-08-24",
  r_pkgs = c("brms", "palmerpenguins", "knitr", "rmarkdown"),
  git_pkgs = list(
    package_name = "cmdstanr",
    repo_url     = "https://github.com/stan-dev/cmdstanr",
    commit       = "da99e2ba954658bdad63bffb738c4444c33a4e0e"
  ),
  system_pkgs = c("quarto", "cmdstan"),
  ide          = "none",
  project_path = ".",
  overwrite    = TRUE
)
```

`r_pkgs`: CRAN packages, at their versions on that date. `knitr` and `rmarkdown` are here because Quarto needs them to run R code.

# Nix/`Rix`: the `rix()` call

```
library(rix)

rix(
  date   = "2026-08-24",
  r_pkgs = c("brms", "palmerpenguins", "knitr", "rmarkdown"),
  git_pkgs = list(
    package_name = "cmdstanr",
    repo_url     = "https://github.com/stan-dev/cmdstanr",
    commit       = "da99e2ba954658bdad63bffb738c4444c33a4e0e"
  ),
  system_pkgs = c("quarto", "cmdstan"),
  ide          = "none",
  project_path = ".",
  overwrite    = TRUE
)
```

`git_pkgs`: packages that aren't on CRAN. cmdstanr is pinned to an exact commit (the v0.9.0 release), not "whatever is on GitHub today".

# Nix/`Rix`: the `rix()` call

```
library(rix)

rix(
  date   = "2026-08-24",
  r_pkgs = c("brms", "palmerpenguins", "knitr", "rmarkdown"),
  git_pkgs = list(
    package_name = "cmdstanr",
    repo_url     = "https://github.com/stan-dev/cmdstanr",
    commit       = "da99e2ba954658bdad63bffb738c4444c33a4e0e"
  ),
  system_pkgs = c("quarto", "cmdstan"),
  ide          = "none",
  project_path = ".",
  overwrite    = TRUE
)
```

`system_pkgs`: software outside R. Quarto and CmdStan are the two things that broke in the demo; now they’re part of the environment.

# Nix/`Rix`: the `rix()` call

```
library(rix)

rix(
  date   = "2026-08-24",
  r_pkgs = c("brms", "palmerpenguins", "knitr", "rmarkdown"),
  git_pkgs = list(
    package_name = "cmdstanr",
    repo_url     = "https://github.com/stan-dev/cmdstanr",
    commit       = "da99e2ba954658bdad63bffb738c4444c33a4e0e"
  ),
  system_pkgs = c("quarto", "cmdstan"),
  ide          = "none",
  project_path = ".",
  overwrite    = TRUE
)
```

`ide`, `project_path`, `overwrite`: no IDE (`"rstudio"`, `"positron"`, `"code"` also work); write `default.nix` into this folder.

# Nix/`Rix`: Python too

```
rix(
  date    = "2026-08-24",
  r_pkgs  = c("brms", "reticulate"),
  py_conf = list(
    py_version = "3.12",
    py_pkgs    = c("pandas", "numpy")
  ),
  ide          = "none",
  project_path = ".",
  overwrite    = TRUE
)
```

```
$ nix-shell --pure --run 'python --version'
Python 3.12.14   # pandas 3.0.4, numpy 2.5.1 (2026-08-24 snapshot)
```

`py_conf`: one `default.nix` pins R, Python *and* both sets of packages. R reaches that exact Python through `reticulate`, never a system one.

# Nix/`Rix`: using it

Install Nix (macOS, Linux, or Windows via WSL2), using the Determinate Systems installer:

```
curl --proto '=https' --tlsv1.2 -sSf -L https://install.determinate.systems/nix | \
  sh -s -- install
```

```
1   Rscript gen-env.R    # rix() writes default.nix
2   nix-build            # build it (slow only the first time)
3   nix-shell --pure --run "quarto render nix_rix_example.qmd"
```

- Plain `nix-shell` drops you into the environment to work interactively
- `--pure`: nothing from your own machine leaks in: not your R, not your packages, not `~/.cmdstan`

# Illustration

github.com/jgeller112/Rix-Nix_Test

# Nix/`Rix`: using it (recording)

# Nix not perfect

- Misdeclared dependencies
  - → A package lists a dependency under Imports that belongs in Suggests
- macOS
  - → The recipe stays valid, but an OS update can break today's build
  - → Recommendation: build inside Docker
- Isolation
  - → Use `nix-shell --pure`: only the declared environment, nothing from the host
- Windows
  - → nix can be cumbersome on Windows: you need WSL

# What a Lab Can Actually Do

Not ready for Nix/{Rix}?

Start smaller:

- {renv} and {rig} for R dependency snapshots and R version management
  → {groundhog}
- Docker for a shared environment
  → Code Ocean is also a good alternative
- {targets} for analytic pipelines
- Metacheck (https://www.scienceverse.org/metacheck_book/) allows for a reproduibility check

# Culture, Not Just Tools

- Train RAs
- Make “can a stranger rerun this?” a lab norm
- Include tools/discussion in courses

# Takeaways

1. Reproducibility is a ladder, not a cliff
   - Every step you take makes your pipeline more reproducible
   - Document everything!
2. Freeze the environment
   - Nix/`{rix}` declares R, packages, and system tools in one file, pinned to a date
3. Science is hard!
   - Try your best!

# Thanks!

- Interested in learning more about Nix and {rix} check out my tutorial paper with Felipe Vieira and Bruno Rodrigues:

**i** Forthcoming in *AMPPS*

Vieira, F. F., Geller, J., & Rodrigues, B. (2026, January 28). Why risk it, when you can {rix} it: A tutorial for computational reproducibility focused on simulation studies. Retrieved from osf.io/preprints/psyarxiv/a3e8v_v1.